



NOMBRE DEL ESTUDIANTE:

Francisco Xavier Gil Ginez

DOCENTE:

José Miguel Carrera Pacheco

MATERIA:

Desarrollo Web Profesional

CUATRIMESTRE ENERO-ABRIL

2026

Recuperación de Contraseñas Accesible y Segura

1. ¿Qué es un token de recuperación?

Un token de recuperación es una cadena de caracteres única, aleatoria y temporal que se genera por el sistema cuando un usuario solicita restablecer su contraseña. Su función principal es actuar como una credencial de un solo uso que permite identificar y autenticar al usuario sin necesidad de conocer su contraseña actual.

Técnicamente, un token de recuperación bien diseñado debe cumplir con las siguientes características:

- Unicidad: cada token debe ser diferente para cada solicitud, incluso si el mismo usuario solicita recuperación múltiples veces.
- Aleatoriedad criptográfica: se genera usando un generador de números pseudoaleatorios criptográficamente seguro (CSPRNG), como `crypto.randomBytes()` en Node.js o `os.urandom()` en Python.
- Longitud suficiente: se recomienda al menos 32 bytes (256 bits) de entropía, lo que lo hace resistente a ataques de fuerza bruta.
- Almacenamiento seguro: no se almacena el token en texto plano en la base de datos, sino su hash (por ejemplo, con SHA-256), igual que una contraseña.

El flujo típico es el siguiente: el usuario solicita la recuperación → el sistema genera el token → se envía al correo registrado del usuario → el usuario hace clic en el enlace con el token → el sistema valida el token → si es válido, permite el cambio de contraseña.

2. ¿Por qué deben expirar los tokens?

La expiración de los tokens de recuperación es una medida de seguridad fundamental. Un token que no expira representa un vector de ataque permanente: si un atacante obtiene acceso al token (por ejemplo, interceptando el correo electrónico, accediendo al

historial del navegador o al servidor de correo), puede usarlo en cualquier momento futuro para tomar control de la cuenta.

Las razones específicas por las que los tokens deben expirar son:

- Minimizar la ventana de ataque: un token válido por 15 o 30 minutos limita el tiempo en que un atacante puede explotarlo.
- Reducir el riesgo de acceso no autorizado posterior: si el correo del usuario es comprometido días después de la solicitud, el token ya no será válido.
- Forzar que el usuario complete el proceso en tiempo razonable: evita solicitudes olvidadas que quedan pendientes indefinidamente.
- Cumplimiento de estándares de seguridad: organizaciones como OWASP recomiendan expiración corta (entre 10 y 60 minutos) para tokens de recuperación.

Adicionalmente, el token debe invalidarse inmediatamente después de ser utilizado una vez, de modo que no pueda reutilizarse aunque no haya expirado todavía. Esto se conoce como tokens de un solo uso (one-time tokens).

3. ¿Qué ataques existen en flujos de recuperación?

El flujo de recuperación de contraseña es uno de los puntos más atacados en aplicaciones web porque, si es vulnerable, permite a un atacante tomar control de cuentas sin conocer la contraseña original. Los principales ataques son:

Account Enumeration (Enumeración de cuentas)

Ocurre cuando el sistema responde de forma diferente si el correo existe o no. Por ejemplo, si el sistema muestra "El correo no está registrado", un atacante puede descubrir qué cuentas existen y usarlas para ataques dirigidos o phishing. La contramedida es mostrar siempre el mismo mensaje genérico, como: "Si el correo está registrado, recibirás instrucciones."

Token Predictable (Token predecible)

Si el token se genera usando funciones no criptográficas (como `Math.random()` en JavaScript o `time()` en C), un atacante puede predecir o reproducir el token. La solución es siempre usar un CSPRNG para generación de tokens.

Token Reuse (Reutilización de token)

Si el sistema no invalida el token tras su uso, un atacante que intercepte el enlace puede reutilizarlo para cambiar la contraseña nuevamente. El token debe eliminarse o marcarse como usado en cuanto se completa el proceso.

Token Leakage (Fuga del token)

El token puede filtrarse a través de: cabeceras HTTP Referer si el enlace es procesado por un servidor externo, historial del navegador, logs del servidor que registran URLs completas, o caché de proxies. La contramedida incluye usar tokens en el cuerpo POST en lugar de en la URL cuando sea posible, y configurar el servidor para no registrar tokens en logs.

Brute Force (Fuerza bruta)

Si los tokens son cortos o predecibles, un atacante puede probar sistemáticamente todas las combinaciones posibles. La solución es combinar tokens largos y aleatorios con límites de intentos (rate limiting) y bloqueo tras múltiples fallos.

Race Condition (Condición de carrera)

Si el sistema no maneja correctamente la concurrencia, un atacante podría realizar múltiples solicitudes simultáneas para explotar ventanas de tiempo donde el token es válido pero aún no ha sido marcado como usado. Se mitiga usando transacciones atómicas en la base de datos al validar y consumir el token.

4. ¿Cómo se diseña un flujo accesible para recuperación de acceso?

Un flujo de recuperación accesible debe equilibrar seguridad con usabilidad, asegurando que todos los usuarios, incluidos quienes tienen discapacidades visuales, motoras o cognitivas, puedan completar el proceso sin barreras.

Principios de accesibilidad aplicados al flujo de recuperación:

Mensajes claros y no reveladores

Los mensajes de error y confirmación deben ser claros para el usuario legítimo pero no revelar información sensible a posibles atacantes. Por ejemplo, en lugar de "Error: token expirado hace 5 minutos", se debe mostrar "El enlace de recuperación ya no es válido. Por favor, solicita uno nuevo." Estos mensajes deben ser accesibles para lectores de pantalla usando atributos ARIA correctos (`role="alert"`, `aria-live="polite"`).

Formularios accesibles

Todos los campos del formulario deben tener etiquetas (label) correctamente asociadas, no solo placeholders. Los campos de nueva contraseña deben contar con indicadores de fortaleza visibles y con descripciones textuales (no solo por color), y deben ser navegables por teclado. El campo de confirmación de contraseña debe validar en tiempo real e informar con un mensaje de texto, no solo con un icono.

Alternativas y canales múltiples

Para usuarios que no tienen acceso a su correo electrónico, el sistema puede ofrecer alternativas como: código de recuperación enviado por SMS, preguntas de seguridad (aunque este método se considera menos seguro), o contacto con soporte humano verificado. Cada alternativa debe ser igualmente accesible.

Tiempos de expiración razonables

El sistema debe considerar que algunos usuarios pueden tardar más en completar el proceso. Una expiración de 30 minutos es un balance adecuado entre seguridad y usabilidad. Si el token expira, el sistema debe permitir solicitar uno nuevo fácilmente, sin bloquear al usuario.

Comunicación por correo electrónico accesible

El correo de recuperación debe ser legible por lectores de pantalla, con texto alternativo en imágenes, jerarquía de encabezados correcta, y un enlace de texto plano como respaldo al botón visual. Se debe incluir también la URL completa en texto para usuarios que no puedan hacer clic en enlaces.

Referencias

- OWASP. (2023). Forgot Password Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html
- OWASP. (2023). Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- NIST. (2020). Digital Identity Guidelines (SP 800-63B). National Institute of Standards and Technology.
- Web Accessibility Initiative (WAI). (2023). WCAG 2.1 Guidelines. W3C. <https://www.w3.org/WAI/WCAG21/quickref/>
- Mozilla Developer Network. (2024). Web Cryptography API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API